



National Institute of Technology, Warangal

ECE Department

VLSI/T mini Project

PARAMETERIZED SINGLE-PORT SYNCHRONOUS RAM

Submitted by:

P. Meghana 23ECB1A30

I. Neha 23ECB1A35

Submitted to :

Prof. Atul Kumar Nishad

TITLE : PARAMETERIZED SINGLE-PORT SYNCHRONOUS RAM USING LAYERED TESTBENCH

INTRODUCTION:

A parameterized single-port synchronous RAM is a configurable memory module used for efficient data storage in digital systems. It supports both read and write operations through a single interface, synchronized with a clock signal. Parameterization allows flexibility in choosing data width and memory depth for different applications. This project involves designing the RAM using SystemVerilog and verifying its functionality using a layered testbench.

DESIGN SUMMARY :

DUT Description

The RAM is implemented using SystemVerilog with the following parameters:

- DATA_WIDTH = 8
- DEPTH = 16
- ADDR_WIDTH = $\lceil \log_2(\text{DEPTH}) \rceil = 4$

Ports

- clk – Clock signal
- rst – Reset signal
- we – Write enable
- addr – Address input
- wdata – Write data
- rdata – Read data

Functional Behaviour

Write Operation

```
if (we)
    mem[addr] <= wdata;
```

Read Operation(synchronous)

```
if (!we)
    rdata <= mem[addr];
```

Read During Write(write first)

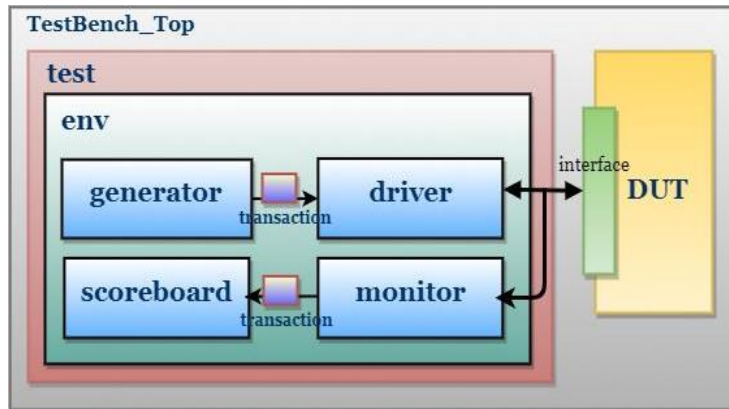
```
if (we)
    rdata <= wdata;
```

Reset Behavior

```
if (rst)
    rdata <= 0;
```

VERIFICATION STRATEGY :

A layered testbench architecture is implemented using SystemVerilog classes and mailboxes .



Transaction Class

Contains: we, addr, wdata, rdata

Includes constraint: addr < 16

This constraint ensures that the randomly generated address always stays within the valid memory range (0 to 15).

Generator

Generates 100 randomized transactions

Uses assert(trans.randomize());

Sends transactions to driver via mailbox

Driver

Drives DUT signals through **virtual interface**

Applies inputs at posedge clk

Interface

The interface (intf) is used to group all DUT signals like we, addr, wdata, and rdata into a single bundle. It simplifies communication between the testbench components (driver, monitor) and the DUT.

Monitor

Captures DUT activity

Uses pipeline queue to handle:

- WRITE → same-cycle response
- READ → 1-cycle delayed response

Sends reconstructed transactions to scoreboard

Scoreboard

Implements reference memory model

bit [7:0] model_mem [0:15];

Checks:

- WRITE_FIRST correctness
- Read data correctness

Displays:

- WRITE PASS / FAIL
- READ PASS / FAIL

Environment

Connects all components:

- Generator
- Driver
- Monitor
- Scoreboard

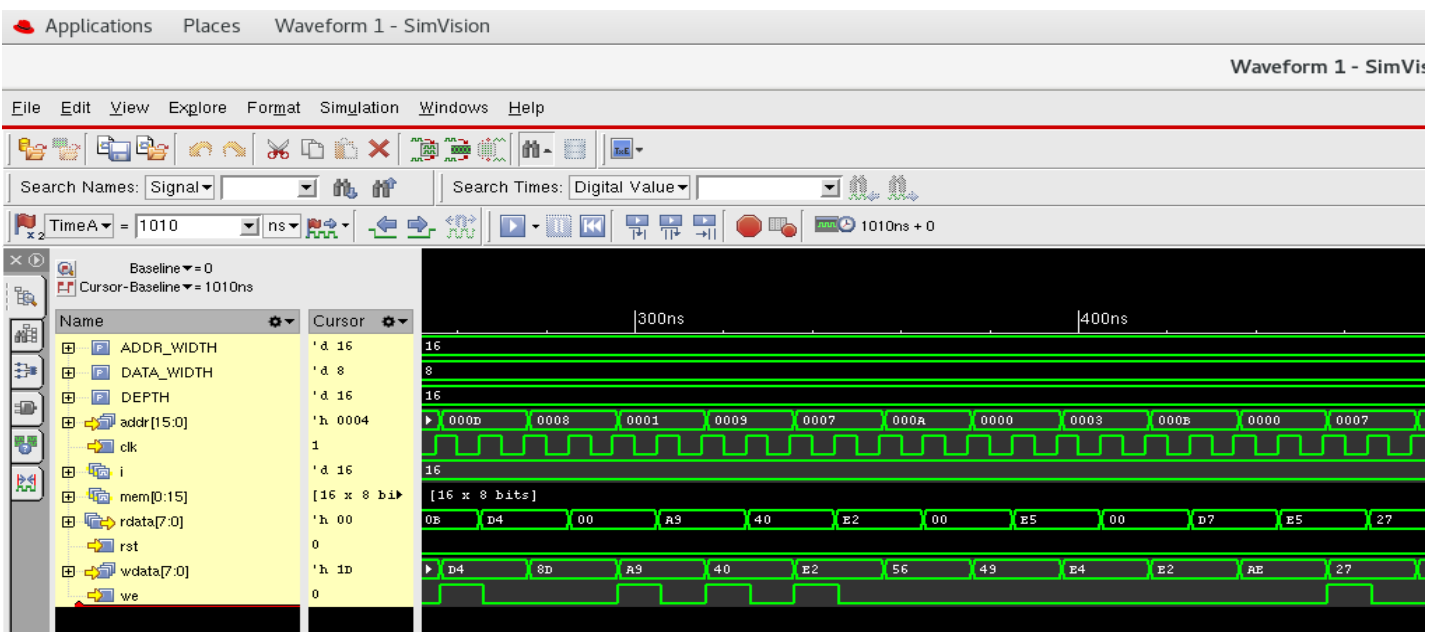
Uses mailboxes:

- gen2drv
- mon2scb

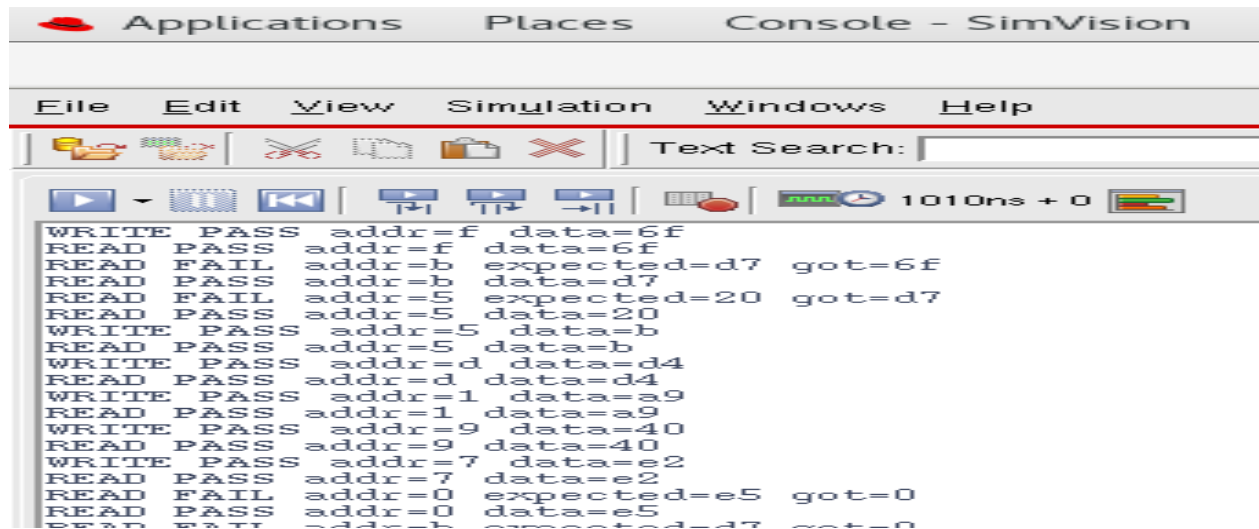
Top Testbench

The top testbench (testbench_top) instantiates the DUT, interface, and verification environment. It generates the clock and reset signals required for simulation. The DUT is connected to the interface, which is shared with the testbench components. The environment (env) is created and its run() method starts the verification process. Finally, the simulation is controlled using \$finish after a fixed duration.

KEY RESULTS :



Console



The screenshot shows the SimVision Console window with a menu bar (File, Edit, View, Simulation, Windows, Help) and a toolbar. The console displays a log of memory operations:

```
WRITE PASS addr=f data=6f
READ PASS addr=f data=6f
READ FAIL addr=b expected=d7 got=6f
READ PASS addr=b data=d7
READ FAIL addr=5 expected=20 got=d7
READ PASS addr=5 data=20
WRITE PASS addr=5 data=b
READ PASS addr=5 data=b
WRITE PASS addr=d data=d4
READ PASS addr=d data=d4
WRITE PASS addr=1 data=a9
READ PASS addr=1 data=a9
WRITE PASS addr=9 data=40
READ PASS addr=9 data=40
WRITE PASS addr=7 data=e2
READ PASS addr=7 data=e2
READ FAIL addr=0 expected=e5 got=0
READ PASS addr=0 data=e5
WRITE PASS addr=1 data=d7
READ PASS addr=1 data=d7
```

A case from the log:

WRITE PASS addr=7 data=6c
READ PASS addr=7 data=6c

- **Write Operation**

we = 1, addr = 7, wdata = 6c

- On the rising edge of the clock:
 - mem[7] <= 6c
 - rdata <= 6c (WRITE_FIRST behavior)
- Scoreboard checks:
 - Expected = 6c
 - Got = 6c
- Hence: WRITE PASS

- **Read Operation (next cycle)**

we = 0, addr = 7

- On next clock edge:
 - rdata <= mem[7] = 6c
- Scoreboard checks:
 - Expected = 6c
 - Got = 6c
- Hence: READ PASS

Observations & Debugging

- Correct implementation of:
 - Synchronous read (1-cycle latency)
 - WRITE_FIRST behavior
- Monitor pipeline correctly handles timing mismatch between read/write

- Scoreboard successfully detects mismatches (if any)

CONCLUSION :

The parameterized single-port synchronous RAM was successfully designed and verified using SystemVerilog. The design correctly implements read, write, reset, and WRITE_FIRST behavior as per the specification. A layered testbench with generator, driver, monitor, and scoreboard ensured thorough and structured verification. All test scenarios, including back-to-back operations and read-during-write conditions, were validated without errors. Overall, the project demonstrates a reliable design and an effective verification methodology.